

PL-SQL Lab1

MANDALA RUTHIK REDDY (AM.SC.U4AIE23129)

QUESTION 1:

a) ODD OR EVEN:

```
1 create or replace function oore(int )
2 returns varchar as
3 $$
4 begin
5
6 if $1%2=0 then
7 return 'Even';
8
9 else
10 return 'Odd';
11
12 end if;
13
14 End;
15
16 $$
17
18 language 'plpgsql';
19
20 select oore(57692346);
```

Data Output Explain Messages Notification

Data Output		Explain	Messages	Notification
oore character varying				
1	Even			

b) SUM:

```
1 create or replace function SUM1(a int, b int )
2 returns int as
3 $$
4 begin
5 return a+b;
6 End;
7
8 $$
9
10 language 'plpgsql';
11
12 select SUM1(5,99);
```

Data Output Explain Messages

Data Output		Explain	Messages
sum1 integer			
1	104		

c) Emp_salary:

```
create or replace function emp_sal (empno emp.eno%type)
returns emp.sal%type as
$$
declare
salary emp.sal%type;
begin
select sal into salary from emp where eno=empno;
return salary;
End;
$$
language 'plpgsql';

select emp_sal(103);
```

emp_sal	
numeric	
1	55000.00

d) Number of Employees:

```
create or replace function noe(deptno dept.dno%type)
returns int as
$$
begin
return (select count(eno) from emp where dno=deptno);
End;
$$
language 'plpgsql';

select noe(10);
```

Data Output		Exp
noe		
integer		
1	2	

e) Salary hike:

```

create or replace function hike (empno emp.eno%type)
returns void as
$$
declare
salary emp.sal%type;
avg_sal emp.sal%type;
depno emp.dno%type;

begin

select dno into depno from emp where eno=empno;
select sal into salary from emp where eno=empno;
select avg(sal) into avg_sal from emp where dno=depno;

if avg_sal>salary then
update emp set sal=salary+0.1*salary where eno=empno;
end if ;
end;
$$
language 'plpgsql';

select hike(103);
select*from emp;

```

f) Dep_name

```

create or replace function dep_name (empname emp.ename%type)
returns dept.dname%type as
$$
begin
return (select dname from dept d, emp e where d.dno=e.dno and ename=empname);
end;
$$
language 'plpgsql';

select dep_name('Bob');

```

Data Output	Explain	Messages	Notificatic
dep_name character varying			
1	Marketing		

QUESTION_2:

a) Total numbers worked on all projects

```

create or replace function ProjectLoad (emno Emp.eno%type)
returns EMP_PROJ.prjt_hrs%type as
$$
declare
hrs EMP_PROJ.prjt_hrs%type;
begin
select sum(prjt_hrs) into hrs from EMP_PROJ where eno=emno;
return hrs;
end;
$$
language 'plpgsql';

```

OUTPUT:

	projectload numeric 
1	350.50

b) Update sal is sal < avg(sal)

```

create or replace function ups (emno Emp.eno%type)
returns void as
$$
declare
salary Emp.sal%type;
begin
select sal into salary from Emp where eno=emno;
if salary<(select avg(sal) from Emp ) then
salary=salary+ (select avg(sal) from Emp )-salary ;
update Emp set sal=salary where eno=emno;
return;
end if;
end;
$$
language 'plpgsql';

select ups(103);

```

OUTPUT:

Question-3:

Consider the following tables Item(ino, iname, unit_price) Transaction(tr_no,ino,qty) Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

Answer:

CREATE OR REPLACE FUNCTION delete_item_if_few_transactions(p_ino INTEGER)

RETURNS VOID AS \$\$

DECLARE

transaction_count INTEGER;

BEGIN

-- Count the number of transactions for the given item

SELECT COUNT(*) INTO transaction_count

FROM Transaction

WHERE ino = p_ino;

-- If fewer than 2 transactions, delete the item

IF transaction_count < 2 THEN

```

DELETE FROM Item
WHERE ino = p_ino;

RAISE NOTICE 'Item % deleted due to insufficient transactions (%).', p_ino, transaction_count;
ELSE
    RAISE NOTICE 'Item % retained. Transactions: %.', p_ino, transaction_count;
END IF;
END;
$$ LANGUAGE plpgsql;

SELECT delete_item_if_few_transactions(103);

```

QUESTION 4:

```

A)
create or replace function update_customer_type(threshold numeric, cust integer)
returns void as $$
declare
    total_bal numeric;
    new_type char(1);
begin
    select sum(bal) into total_bal from accounts where cust_no = cust;

    if total_bal is null then
        raise notice 'customer has no accounts';
        return;
    end if;

    if total_bal > threshold then
        new_type := 'a';
    else
        new_type := 'b';
    end if;
end;

```

end if;

update customer set c_type = new_type where cno = cust;

end;

\$\$ language plpgsql;

B)

create or replace function closebranch(old_branch integer, new_branch integer)

returns void as \$\$

begin

update accounts set br_no = new_branch where br_no = old_branch;

delete from branches where br_no = old_branch;

raise notice 'branch % closed', old_branch;

end;

\$\$ language plpgsql;

C)

create or replace function safe_withdraw(ac integer, amount numeric)

returns text as \$\$

declare

current_bal numeric;

begin

select bal into current_bal from accounts where ac_no = ac;

if current_bal is null then

return 'account not found';

end if;

if current_bal < amount then

```
        return 'insufficient funds';
    end if;

    update accounts set bal = bal - amount where ac_no = ac;

    return 'withdrawal successful';
end;
$$ language plpgsql;

select safe_withdraw(1001,200);
```

	safe_withdraw text	
1	withdrawal successful	

2. Consider the following schema .
Emp(eno, ename, sal)
EMP_PROJ(eno, pno, prjt_hrs).

- Write a function ProjectLoad that returns the total project working hours for the given eno.
-
- Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

New salary is current sal + difference between current sal and average salary

```
-- (Tables required for the program)
DROP TABLE IF EXISTS emp_proj;
DROP TABLE IF EXISTS emp;

CREATE TABLE emp (
  eno INT PRIMARY KEY,
  ename VARCHAR(50),
  sal NUMERIC
);

CREATE TABLE emp_proj (
  eno INT REFERENCES emp(eno),
  pno INT,
  prjt_hrs INT
);

INSERT INTO emp VALUES
(101, 'Ravi', 20000),
(102, 'Kiran', 30000),
(103, 'Sita', 15000);

INSERT INTO emp_proj VALUES
(101, 1, 10),
(101, 2, 20),
(102, 1, 5),
(103, 1, 8);

-- (a) Function ProjectLoad
CREATE OR REPLACE FUNCTION ProjectLoad(p_eno INT)
RETURNS INT AS $$
DECLARE
  total_hrs INT := 0;
BEGIN
  SELECT COALESCE(SUM(prjt_hrs), 0)
  INTO total_hrs
  FROM emp_proj
  WHERE eno = p_eno;

  RETURN total_hrs;
END;
$$ LANGUAGE plpgsql;

-- (b) Update salary if less than average
DO $$
DECLARE
  avg_sal NUMERIC;
BEGIN
  SELECT AVG(sal) INTO avg_sal FROM emp;

  UPDATE emp
  SET sal = sal + (avg_sal - sal)
  WHERE sal < avg_sal;
END;
$$;
```

```
-- Output
SELECT 'Total Project Hours for 101 = ' || ProjectLoad(101) AS output;
SELECT * FROM emp;
```

Output:

```
DROP TABLE
DROP TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 3
INSERT 0 4
CREATE FUNCTION
DO
```

output

```
-----
Total Project Hours for 101 = 30
(1 row)
```

```
eno | ename |      sal
-----+-----+-----
102 | Kiran |      30000
101 | Ravi  | 21666.666666666667
103 | Sita  | 21666.666666666667
(3 rows)
```

```
psql:commands.sql:2: NOTICE:  table "emp_proj" does not exist, skipping
psql:commands.sql:3: NOTICE:  table "emp" does not exist, skipping
```

3. Consider the following tables
Item(ino, iname, unit_price)
Transaction(tr_no,ino,qty)

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
DROP TABLE IF EXISTS transaction;
DROP TABLE IF EXISTS item;

CREATE TABLE item (
    ino INT PRIMARY KEY,
    iname VARCHAR(50),
    unit_price NUMERIC
);

CREATE TABLE transaction (
    tr_no INT PRIMARY KEY,
    ino INT REFERENCES item(ino) ON DELETE CASCADE,
    qty INT
);

INSERT INTO item VALUES
(1, 'Pen', 10),
(2, 'Book', 50),
(3, 'Pencil', 5);

INSERT INTO transaction VALUES
(101, 1, 2),
(102, 1, 3),
(103, 2, 1);

-- Function to delete item if transactions < 2
CREATE OR REPLACE FUNCTION CheckAndDeleteItem(p_ino INT)
RETURNS TEXT AS $$
DECLARE
    trans_count INT := 0;
BEGIN
    SELECT COUNT(*) INTO trans_count
    FROM transaction
    WHERE ino = p_ino;

    IF trans_count < 2 THEN
        DELETE FROM item WHERE ino = p_ino;
        RETURN 'Item deleted';
    ELSE
        RETURN 'Item NOT deleted - transactions >= 2';
    END IF;
END;
$$ LANGUAGE plpgsql;

-- Output
SELECT CheckAndDeleteItem(2);
SELECT * FROM item;
```

Output:

```
DROP TABLE
DROP TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 3
INSERT 0 3
CREATE FUNCTION
  checkanddeleteitem
-----
Item deleted
(1 row)

  ino |  iname  | unit_price
-----+-----+-----
    1 | Pen    |         10
    3 | Pencil |          5
(2 rows)
```

4.

Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)

BRANCHES(br_no, br_name, loc)

CUSTOMER(cno, cname, c_type)

- Write a function that accepts a threshold value and a customer number . The program updates the c_type based on the threshold value. Ie. If balance> threshold then class A, else class B.
- Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.
- Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```
(...and the program)

DROP TABLE IF EXISTS accounts;
DROP TABLE IF EXISTS branches;
DROP TABLE IF EXISTS customer;

CREATE TABLE branches (
    br_no INT PRIMARY KEY,
    br_name VARCHAR(50),
    loc VARCHAR(50)
);

CREATE TABLE customer (
    cno INT PRIMARY KEY,
    cname VARCHAR(50),
    c_type VARCHAR(10)
);

CREATE TABLE accounts (
    ac_no INT PRIMARY KEY,
    br_no INT REFERENCES branches(br_no) ON DELETE CASCADE,
    cust_no INT REFERENCES customer(cno) ON DELETE CASCADE,
    ac_type VARCHAR(20),
    bal NUMERIC
);

INSERT INTO branches VALUES
(1, 'Main', 'Hyd'),
(2, 'City', 'Hyd'),
(3, 'Uptown', 'Hyd');

INSERT INTO customer VALUES
(101, 'Ravi', 'B'),
(102, 'Kiran', 'B'),
(103, 'Sita', 'B');

INSERT INTO accounts VALUES
(1001, 1, 101, 'Savings', 50000),
(1002, 1, 102, 'Savings', 20000),
(1003, 2, 103, 'Current', 80000);
```

```

-- (a) Function to update customer c_type based on threshold
CREATE OR REPLACE FUNCTION UpdateCustomerType(p_threshold NUMERIC, p_cust INT)
RETURNS TEXT AS $$
DECLARE
    total_bal NUMERIC := 0;
BEGIN
    SELECT COALESCE(SUM(bal),0)
    INTO total_bal
    FROM accounts
    WHERE cust_no = p_cust;

    IF total_bal > p_threshold THEN
        UPDATE customer SET c_type = 'A' WHERE cno = p_cust;
        RETURN 'Customer upgraded to Class A';
    ELSE
        UPDATE customer SET c_type = 'B' WHERE cno = p_cust;
        RETURN 'Customer set to Class B';
    END IF;
END;
$$ LANGUAGE plpgsql;

-- (b) Function CloseBranch(old_branch, new_branch)
CREATE OR REPLACE FUNCTION CloseBranch(p_old INT, p_new INT)
RETURNS TEXT AS $$
BEGIN
    UPDATE accounts
    SET br_no = p_new
    WHERE br_no = p_old;

    DELETE FROM branches WHERE br_no = p_old;

    RETURN 'Branch closed and accounts transferred';
END;
$$ LANGUAGE plpgsql;

-- (c) Safe withdrawal function
CREATE OR REPLACE FUNCTION SafeWithdraw(p_acno INT, p_amt NUMERIC)
RETURNS TEXT AS $$
DECLARE
    cur_bal NUMERIC := 0;
BEGIN
    SELECT bal INTO cur_bal FROM accounts WHERE ac_no = p_acno;

    IF cur_bal IS NULL THEN
        RETURN 'Account does not exist';
    END IF;

    IF cur_bal < p_amt THEN
        RETURN 'Insufficient funds - Withdrawal denied';
    ELSE
        UPDATE accounts
        SET bal = bal - p_amt
        WHERE ac_no = p_acno;
        RETURN 'Withdrawal successful';
    END IF;
END;
$$ LANGUAGE plpgsql;

```

```

SELECT UpdateCustomerType(40000, 101);
SELECT CloseBranch(2, 1);
SELECT SafeWithdraw(1001, 10000);
SELECT * FROM accounts;
SELECT * FROM customer;
SELECT * FROM branches;

```

```
CREATE FUNCTION
```

```
CREATE FUNCTION
```

```
CREATE FUNCTION
```

```
    updatecustomertype
```

```
-----
```

```
    Customer upgraded to Class A
```

```
(1 row)
```

```
    closebranch
```

```
-----
```

```
    Branch closed and accounts transferred
```

```
(1 row)
```

```
    safewithdraw
```

```
-----
```

```
    Withdrawal successful
```

```
(1 row)
```

```
ac_no | br_no | cust_no | ac_type | bal
```

```
-----+-----+-----+-----+-----
```

```
1002 | 1 | 102 | Savings | 20000
```

```
1003 | 1 | 103 | Current | 80000
```

```
1001 | 1 | 101 | Savings | 40000
```

```
(3 rows)
```

```
cno | cname | c_type
```

```
-----+-----+-----
```

```
102 | Kiran | B
```

```
103 | Sita | B
```

```
101 | Ravi | A
```

```
(3 rows)
```

```
br_no | br_name | loc
```

```
-----+-----+-----
```

```
1 | Main | Hyd
```

```
3 | Uptown | Hyd
```

```
(2 rows)
```